

Where should the spikes go?

A liquid–spiking, language-conditioned drone policy with verify-behind speculative control, and an operation-level energy analysis

Laalini Bhogadi

[Affiliation]

laalinib68@gmail.com

Abstract

Neuromorphic and continuous-time networks are attractive for onboard robot control because they compute sparsely and integrate irregularly sampled sensor streams natively. We present PLM, a language-conditioned drone policy whose core combines leaky integrate-and-fire (LIF) spiking self-attention, with a leak driven by the *measured* inter-sample interval Δt , and a closed-form liquid time-constant (LTC) recurrence. PLM grounds camera and LiDAR/4D-radar observations in a shared bird’s-eye-view grid, is trained with a four-stage curriculum (cross-modal alignment, a residual latent world model with spike-rate regularisation, chunked imitation, and drafter co-training), and runs under *verify-behind* speculative control: a camera-free spiking drafter acts on every tick while the full model verifies batches of executed actions off the critical path, behind a control-barrier-function safety filter. An operation-level energy analysis, using measured firing rates, gives the central neuromorphic result. When only the attention products are spike-driven, as in spiking-transformer designs, they account for 1.8% of the core’s operations in a short control window, so spiking saves 1.8% of the core’s energy even at a firing rate of 0.079. Making the *inputs to the projections* spike-driven covers 58.9% of the core’s operations and saves 58.0%; its offline error and closed-loop success (0.0081 chunk MSE, 0.12 success) stay within the variation we measure across training seeds. At the system level, the camera encoder takes 88% of per-tick operations, and verify-behind reduces critical-path energy $11\times$ and latency about $6\times$ while matching the full model’s closed-loop success. We also report four failure modes that per-stage offline metrics hid, and we are explicit about the limits of this simulation-only, CPU-scale study.

Keywords: spiking neural networks, liquid time-constant networks, neuromorphic robotics, energy estimation, speculative decoding, vision–language–action models, UAV control

1 Introduction

Onboard control of small aerial robots is bounded by energy and latency. A quadrotor avoiding a moving obstacle at 3 m/s travels 15 cm in 50 ms, and its compute budget is watts, not the hundreds of watts behind today’s language-conditioned robot policies [1–3]. Neuromorphic

computing addresses this with sparse, event-driven computation, in which a synaptic operation is performed only when a spike arrives [4, 5]. Continuous-time networks such as liquid time-constant (LTC) networks [6, 7] integrate irregularly sampled inputs natively and have shown robust out-of-distribution drone flight [8, 9].

Spiking transformers [10, 11] brought spikes to attention, and a natural idea is to use them as the temporal core of a control policy. Two questions are rarely answered for such a system as a whole. *First, where do the spikes actually save energy?* A design can be “spiking” while almost all of its operations remain dense. *Second, what does the rest of the system cost?* The perception front-end may dwarf the spiking core. We build a complete language-conditioned policy, PLM, and answer both questions with an operation-level energy analysis at measured firing rates.

Contributions.

1. **A liquid–spiking control core** with a Δt -aware LIF leak, spike-rate regularisation, and a closed-form LTC recurrence over measured, non-uniform sample intervals, embedded in a BEV-grounded, language-conditioned drone policy (§3).
2. **An operation-level energy analysis** (§6, §7.2) showing that spiking only the attention operands saves 1.8% of the core’s energy in a short control window, while spike-driven projection inputs save 58.0% with no accuracy loss detectable across training seeds. It also shows that the perception front-end, not the core, dominates system energy.
3. **Verify-behind speculative control** (§5): a camera-free spiking drafter on the critical path, with batched target verification, a bounded verification lag and CBF safety (Proposition 1). It cuts critical-path energy $11\times$.
4. **Four diagnostic findings** from a staged curriculum (§4): cross-modal alignment erases motion information; a non-residual latent world model does worse than persistence; ego-motion inputs cause a copycat failure; and geometry-only alignment is colour-blind. The last two were invisible to offline metrics.

2 Background and related work

Spiking networks and neuromorphic hardware. Spiking neurons communicate binary events, so a weight is only accumulated when a spike arrives. On neuromorphic processors such as Loihi [5], cost scales with the number of synaptic events rather than with dense matrix size [4]. Surrogate gradients make spiking networks trainable with back-propagation [12, 13]. Spikformer [10] and the spike-driven transformer [11] turn Q, K and V into spike trains so that the attention products become accumulations. Energy in this literature is usually estimated by counting multiply-accumulates (MACs) and accumulates (ACs) with 45 nm figures [14]; we follow that convention.

Continuous-time networks. LTC networks [6] have input-dependent time constants, and closed-form continuous-time (CfC) networks [7] remove the ODE solver. Neural circuit policies [9] and liquid networks [8] have flown drones robustly out of distribution.

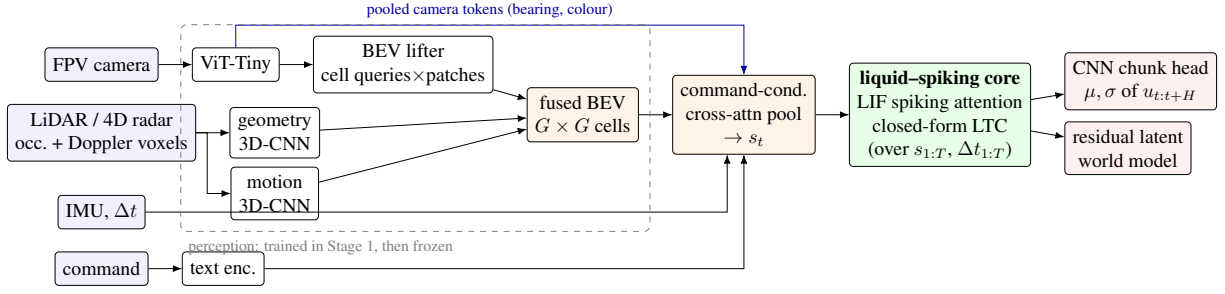


Figure 1: PLM. Perception grounds camera and LiDAR/radar in a metric BEV grid. Command-conditioned pooling produces one token per control tick, and the liquid-spiking core integrates the tokens over the measured Δt . The edge drafter (not shown) uses only radar pillars, altitude, Δt and the command, followed by a one-layer liquid-spiking core.

Language-conditioned policies and action chunking. VLA models [1–3, 15, 16] map images and instructions to actions. Action chunking [17, 18] predicts short trajectories. These systems target server-class compute, whereas we target the edge.

Speculative decoding and safe control. Speculative decoding [19–21] hides a large model’s latency behind a drafter by verifying drafts *before* emitting them. That is impossible when the drafts are actions whose consequences form the next input. Control barrier functions [22] give forward invariance of safe sets and serve here as a shield for unverified actions.

3 PLM architecture

Perception. A ViT-Tiny [23] produces camera patch tokens. G^2 learned bird’s-eye-view (BEV) queries, one per metric cell, cross-attend to them [24] to give camera cell features. LiDAR/4D-radar voxels carry occupancy and radial Doppler velocity. Two 3D-CNN trunks collapse them over height into pillar features [25]: a geometry trunk and a lightweight motion trunk (the reason for two trunks is Finding 1). The policy reads *scene tokens*: the fused BEV cells followed by pooled perspective camera tokens.

Command-conditioned tokenisation. A command embedding offsets n_q learned queries that cross-attend to the scene tokens. The result, plus an embedding of altitude and Δt (Finding 3 explains why ego-motion is excluded), is one token s_t per tick.

Liquid-spiking core. Each of the L layers applies pre-norm causal self-attention over $s_{1:T}$ with LIF spiking Q, K and V, followed by a closed-form LTC recurrence. With measured intervals Δt_t , each LIF unit integrates

$$u_t = \beta_t u_{t-1} + (1 - \beta_t) x_t, \quad \beta_t = e^{-\Delta t_t / \tau}, \quad o_t = \Theta(u_t - \vartheta), \quad u_t \leftarrow u_t - \vartheta o_t, \quad (1)$$

with learnable per-unit τ , soft reset, and the fast-sigmoid surrogate $\partial o / \partial u \approx (1 + k|u - \vartheta|)^{-2}$ [13]. Because the leak uses the real elapsed time, a late sensor packet decays the membrane exactly as physics would, rather than as one fixed “step”. The LTC update is exact for piecewise-constant

input [7]:

$$f_t = \text{softplus}(W_x x_t + W_h h_{t-1} + b), \quad g_t = \tau^{-1} + f_t, \quad h_t = h_{t-1} \odot e^{-\Delta t g_t} + \frac{A \odot f_t}{g_t} \odot (1 - e^{-\Delta t g_t}). \quad (2)$$

h_t is a convex combination of h_{t-1} and $A \odot f_t / g_t$, so it stays bounded for any Δt . For training stability we clip gradients to global norm 1, rescale W_h after every step so that $\|W_h\|_2 \leq 0.99$, use truncated BPTT with detached state, and keep membrane and ODE updates in FP32 under mixed precision.

Spike-driven projections (variant). In the design above, the operands of $QK^\top$ and AV are binary, but the *inputs* to W_{qkv} and to the LTC’s W_x are real-valued layer-normalised activations, so those products remain dense MACs. The `spike_inputs` variant places an additional Δt -aware LIF population ((1)) in front of W_{qkv} and W_x . Their inputs become binary, and those matmuls become accumulate-only.

Spike-rate regularisation. To avoid both quiescence (no spikes, hence no gradient) and hyper-activity, every LIF population’s mean rate $\bar{r}_l$ is regularised towards $r^* = 0.08$ with $\lambda \sum_l (\bar{r}_l - r^*)^2$.

Heads. A 1-D convolutional decoder emits a Gaussian action chunk (forward, lateral and vertical velocity, and yaw rate) over H steps. A residual predictor $\hat{s}_{t+h} = \text{sg}(s_t) + g_\psi(h_t, \omega_t, h)$ forecasts the future fused-BEV latent.

4 Training curriculum and diagnostic findings

Stage 1 (perception). Cell-level radar $\leftrightarrow$ camera InfoNCE [26], camera occupancy, Doppler regression, and BEV and per-token semantics. Perception is then frozen, and every episode is encoded once into an FP16 feature cache. **Stage 2 (world model).** A multi-horizon latent prediction loss with the spike-rate regulariser and truncated BPTT, using no action labels. **Stage 3 (imitation).** A discounted multi-horizon chunk loss, a jerk penalty, a Gaussian NLL on σ , and the Stage-2 loss as an auxiliary, on DART-perturbed demonstrations [27]. **Stage 4 (drafter).** KL distillation [28] from the target with a safety hinge, then retraining on a hard-rejection buffer of states whose drafts the target rejected in closed-loop rollouts.

The staged design exposed four failure modes, each diagnosed with a controlled comparison:

Finding 1: alignment erases motion. With one shared radar trunk, Doppler error stayed at the predict-zero baseline for more than 1,000 steps, although Doppler is an input channel. The same trunk trained on Doppler alone reached 0.074 against a 0.21 baseline. A single camera frame carries no velocity, so cell-level alignment pushes motion out of the shared features. A private motion trunk fixes this (Table 1).

Finding 2: residual world models. A non-residual latent predictor was $12\times$ worse than persistence. The residual form beats persistence (ratio 0.91).

Finding 3: copycat. Ego-velocity explains 97% of the variance of the expert’s forward command. With the full IMU as input, the policy had a chunk error $19\times$ below the mean-action

Algorithm 1 Verify-behind speculative control (one tick)

```
1: encode radar pillars; drafter latent  $z_t$  ▷ critical path
2:  $\hat{u}_t \leftarrow \text{cache}(z_t, \text{cmd})$  or  $\text{drafter}(z_{t-T+1:t}, \Delta t_{t-T+1:t})$  ▷ critical path
3: execute CBF( $\hat{u}_t$ ); append the window and  $\hat{u}_t$  to pending
4: if  $|\text{pending}| \geq k$  then ▷ off the critical path
5:   target on all pending windows in one batched pass  $\rightarrow (\mu_i, \sigma_i)$ 
6:    $\text{accept}_i \leftarrow \forall j : |\hat{u}_{i,j} - \mu_{i,j}| \leq \epsilon + z\sigma_{i,j}$ ; rejects  $\rightarrow$  hard buffer; confident accepts  $\rightarrow$ 
     cache
7:   if the last draft is rejected then override the current action with CBF( $\mu_{\text{last}}$ )
8:    $k \leftarrow 1$  if any rejection, else  $\min(2k, k_{\text{max}})$ 
9: end if
```

baseline and 0% closed-loop success: it copied its own motion [29, 30]. The policy and drafter therefore see only altitude and Δt , and relative motion arrives through radar Doppler.

Finding 4: colour-blind features. Without semantic supervision, all Stage-1 losses were geometric, the frozen features could not tell the red beacon from the blue one, and closed-loop success was 4%. Perspective camera tokens with per-token semantics fixed this (beacon accuracy 0.93).

5 Verify-behind speculative control

In speculative decoding the drafts are verified before they are emitted; in control, verifying the action for step $t+i$ requires an observation that exists only after the earlier actions have been executed. Verify-behind therefore executes drafts immediately and verifies them in batches (Algorithm 1).

Proposition 1 (Bounded verification lag and drafter-independent safety). *(i) Every executed action is compared with the target’s action for the same observation window at most $k_{\text{max}} - 1$ ticks later. (ii) After a rejection, the next action is verified on the same tick. (iii) If the CBF program is feasible at every tick and the dynamics match the filter’s single-integrator model, the safe set $\{x : h_i(x) \geq 0 \forall i\}$ is forward invariant, whatever the drafter outputs.*

Proof sketch. (i) and (ii) follow from the invariant $|\text{pending}| \leq k \leq k_{\text{max}}$ and the rule for updating k . (iii) is the standard CBF result [22]. $\square$

6 Energy accounting

We count the MACs of every matrix multiplication and convolution per control tick with PyTorch’s operation counter, and exclude elementwise operations (LIF and LTC updates, softmax, normalisation), as is conventional. A product with one binary spike operand is re-expressed as accumulates at the *measured* firing rate r : a dense product with M MACs becomes rM ACs. Energy uses the 45 nm figures $E_{\text{MAC}} = 4.6$ pJ and $E_{\text{AC}} = 0.9$ pJ [14], as in [10, 11]. The core is counted over its T -step window (it is re-run on the sliding window every tick), and the action head at the last step only, which is all the controller decodes. These are *estimates for comparing*

Table 1: Per-stage diagnostics on held-out data, each against a trivial baseline.

Stage	Metric	PLM	Baseline
1	cell-level radar $\leftrightarrow$ camera top-1	0.410	0.0072 (chance)
1	camera $\rightarrow$ BEV occupancy IoU	0.452	–
1	beacon accuracy, camera tokens / BEV cells	0.93 / 0.39	0.25 (chance)
1	Doppler MAE	0.048	0.209 (predict 0)
2	latent prediction MSE	0.0343	0.0376 (persistence)
2	mean firing rate	0.077	target 0.08
3	action-chunk MSE	0.0062	0.0803 (mean action)
3	chunk jerk	0.0003	0.0002 (expert)
4	offline draft acceptance	0.762	0.840 (distillation only)

designs, not measurements on neuromorphic hardware: memory traffic and data movement are ignored.

7 Experiments

Setup. We use a procedural quadrotor simulator: first-order velocity tracking, $\Delta t \sim \mathcal{U}[30, 80]$ ms, 6 sphere obstacles of which about 40% move, red and blue beacons, a rendered 96 px FPV camera, $2 \times 8 \times 24 \times 24$ occupancy+Doppler voxels with dropout and clutter, and seven instruction intents with paraphrases. Success means reaching the named beacon within 1.2 m, without collision, within 150 ticks. The CPU-scale configuration uses $d=128$, $L=2$, $T=8$, $H=8$ and 400 training episodes, with 30 evaluation episodes per condition on seeds shared across all conditions. The ViT was trained from scratch because pretrained weights were unavailable in our environment. Parameters: target 6.79 M (of which ViT 5.49 M), drafter 47 k.

7.1 Per-stage diagnostics

Every stage beats its trivial baseline (Table 1).

7.2 Where the spikes are, and what they cost

The core is cheap; the camera is not. Table 2 breaks one control tick into components. The camera ViT accounts for 88% of the full model’s dense operations, the LiDAR/radar pillar encoder for most of the rest, and the liquid-spiking core ($T=8$ steps, two layers) for under 1%. Any claim of neuromorphic efficiency for the *system* therefore depends first on the perception front-end, not on the core. The radar voxel grid is more than 99% empty in our scenes, which makes the pillar encoder a natural candidate for event-driven or sparse convolution.

Where the spikes go matters more than whether a network spikes. Table 3 compares four cores. In the as-built core, the spike trains of Q, K and V turn $QK^\top$ and AV into accumulates. At the measured firing rate of 0.079 these products become almost free, but in a short control window they are only 1.8% of the core’s operations. The projections W_{qkv} , W_x , W_h and the output layers dominate, so the as-built saving over a dense core is 1.8%. Adding Δt -aware LIF populations in front of W_{qkv} and W_x (spike-driven projection inputs) makes 58.9% of the

Table 2: Dense operation count and energy per control tick, by component (MACs at 4.6 pJ, before any spike-driven reduction). The camera encoder dominates; the liquid–spiking core is 0.8% of a full-model tick.

Component	MMACs	μJ
LiDAR/radar pillar encoder	21.22	97.6
camera ViT-Tiny	202.05	929.4
BEV lifter + fusion	2.49	11.4
state tokenizer (1 step)	3.02	13.9
target liquid–spiking core (T steps)	1.84	8.4
target action head (last step)	0.27	1.2
drafter latent (1 step)	0.09	0.4
drafter core (T steps) + head (last step)	0.19	0.9

Table 3: Core variants (Stages 2–3 retrained from the same frozen perception and feature cache; closed loop on the same 30 seeds, 24 goal-reaching). “Spike-driven share” is the fraction of the core’s dense MACs whose inputs are spikes; energy is per control tick (window T) with accumulates at the measured firing rate.

Core	Spike-driven share	Core μJ /tick	Saving vs dense	Rate	Chunk MSE	Success (every tick)	Success (chunk)	Collision
LIF spiking attention (PLM)	1.8%	8.29	1.8%	0.079	0.0062	0.12	0.17	0.00
Dense attention (no LIF)	0.0%	8.44	0.0%	–	0.0074	0.12	0.08	0.00
No LTC (feed-forward MLP)	2.1%	7.09	2.1%	0.079	0.0083	0.04	0.04	0.00
Spike-driven projection inputs	58.9%	3.55	58.0%	0.078	0.0081	0.12	0.04	0.00

core’s operations spike-driven and saves 58.0% of its energy, at a similar firing rate (0.078). The remaining dense operations are the recurrent $W_h h_{t-1}$, the attention output projection and the LTC read-out, whose inputs are real-valued states. Spiking those as well bounds the saving at about 98% for these firing rates, but it would change the continuous-time state that makes the LTC useful, and we leave it to future work.

Accuracy of the variants. Across three training seeds of the as-built model, held-out chunk error is 0.0075 ± 0.0011 and closed-loop success 0.10 ± 0.02 (per seed: 0.12, 0.08, 0.08). Every variant in Table 3 lies within that spread on both measures. We therefore do not claim that the LIF attention or the LTC recurrence *improves* accuracy at this scale. The feed-forward (no-LTC) core has the lowest success and the highest error, but by one episode, which is within noise. The claim we do make is narrower and robust to this spread: *moving the spikes to the projection inputs cuts the core’s estimated energy by more than half without a detectable loss in accuracy.*

7.3 Closed-loop control and verify-behind

Table 4 evaluates the controllers on identical seeds. Verify-behind matches the full model’s closed-loop success and collision rate while its critical path (radar pillars, the drafter latent, and the drafter core on a cache miss) costs $98 \mu\text{J}$ per tick, against $1062 \mu\text{J}$ for running the full model every tick ($11 \times$ less), and 4.1 against 24.2 ms of latency. Total energy is *not* reduced, because the target still verifies every window off the critical path; the gain is in what the control loop waits for. The drafter alone collides in 3% of episodes even with the CBF filter, and adding verification

Table 4: Closed-loop control (30 episodes, 24 goal-reaching; success with 95% Wilson interval). Critical-path latency on a 2-core CPU; energy per tick from operation counts (critical path / total including off-path verification).

Controller	Success	Collision	Jerk	Accept.	Crit. ms	Crit. μJ	Total μJ
Expert (privileged)	1.00 [0.86,1.00]	0.00	0.0004	–	10.4	–	–
Target, every tick	0.12 [0.04,0.31]	0.00	0.0037	–	24.2	1062	1062
Target, chunk ($k=4$)	0.17 [0.07,0.36]	0.00	0.0051	–	18.6	1055	1055
Drafter only	0.08 [0.02,0.26]	0.03	0.0011	–	7.5	99	99
Verify-behind	0.12 [0.04,0.31]	0.00	0.0158	0.75	4.1	98	1062

Table 5: Offline imitation error versus closed-loop outcome (target policy, every tick). Lower offline error did not mean better control.

Run	Chunk MSE	vs. mean action	Success	Final goal dist. (m)
Full IMU to policy (copycat)	0.0042	19.0×	0.00	12.9
Geometry-only perception (colour-blind)	0.0080	10.0×	0.04	12.2
PLM	0.0062	13.0×	0.12	11.7
PLM + DAgger (2 rounds)	0.0062	12.9×	0.12	6.4

removes those collisions. The target accepted 75% of executed drafts. A muscle-memory cache of verified chunks supplied most actions, which shortens the critical path but raises action jerk as the source switches between cache, drafter and override. Latency was measured on a shared 2-core CPU with the action head evaluated over the whole window; the energy figures assume the head is decoded at the last step only, which gives identical actions.

7.4 Offline metrics hide closed-loop failure

Table 5 collects the failure modes from §4. The copycat policy had the *lowest* held-out imitation error of all runs and 0% success; the colour-blind policy had a similar error and 4% success. Only closed-loop evaluation revealed either. Two rounds of DAgger, in which the policy flies and the expert labels the states it actually visits, halved the mean final distance to the goal without raising success. This points to demonstration coverage and expert smoothness as the main limits on absolute performance, rather than the neuromorphic core.

8 Discussion and limitations

Three messages carry over beyond this system. First, for spiking control policies, operation-level accounting should accompany any efficiency claim. A spiking-transformer core can be spiking in name while less than 2% of its operations are spike-driven. Second, the perception front-end dominates a multimodal policy’s energy, so neuromorphic gains at system level require event-driven perception (event cameras, sparse radar/LiDAR processing), not only a spiking core. Third, the continuous-time components integrate the measured Δt at no extra operation cost: both the LIF leak and the closed-form LTC update are elementwise. They also remain stable

under irregular sampling, which matters on real sensor buses even when accuracy gains are not visible in simulation.

Limitations. (1) *Simulation only*: kinematic dynamics, sphere obstacles and untextured rendering; nothing here establishes sim-to-real transfer. (2) *Estimated, not measured, energy*: operation counts with 45 nm constants ignore memory traffic, data movement and hardware-specific costs; deployment on Loihi-class hardware is future work. (3) *Low absolute success*: 12–17% against a privileged expert’s 100%. We trace this to demonstration coverage; DAgger halves the final distance to the goal but does not yet close the gap. (4) *Scale and statistics*: CPU-scale models, a from-scratch 96 px ViT, 3 training seeds for the main model and one per variant, and 30 evaluation episodes per condition, so the intervals are wide and small differences between variants are not significant. (5) *Privileged components*: the expert and the CBF use ground-truth obstacle states. (6) The Gaussian action head is unimodal.

9 Conclusion

Building a complete liquid–spiking control policy and accounting for its operations shows that *where* the spikes sit matters more than whether a network is “spiking”. In a short-horizon control transformer, spiking the attention operands is almost free of energy benefit, because the projections dominate. Spiking the projection inputs is where the savings lie, and at the system level the dense perception front-end, not the core, is the next target. Verify-behind control keeps an expensive model off the critical path without giving up verification, and the staged curriculum’s diagnostics caught four failures that offline metrics would have hidden.

Data availability statement

All code (simulator, model, training curriculum, runtime, energy accounting and the scripts that generate every table in this paper) and the run reports and logs underlying the results will be made openly available in a public repository upon publication. [\[Add repository URL / DOI.\]](#)

Acknowledgements

[\[Optional.\]](#)

References

- [1] Anthony Brohan, Noah Brown, Justice Carbajal, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.
- [2] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024.

- [3] Kevin Black, Noah Brown, Danny Driess, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [4] Kaushik Roy, Akhilesh Jaiswal, and Priyadarshini Panda. Towards spike-based machine intelligence with neuromorphic computing. *Nature*, 2019.
- [5] Mike Davies, Narayan Srinivasa, Tsung-Han Lin, et al. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 2018.
- [6] Ramin Hasani, Mathias Lechner, Alexander Amini, Daniela Rus, and Radu Grosu. Liquid time-constant networks. In *AAAI Conference on Artificial Intelligence*, 2021.
- [7] Ramin Hasani, Mathias Lechner, Alexander Amini, Lucas Liebenwein, Aaron Ray, Max Tschaikowski, Gerald Teschl, and Daniela Rus. Closed-form continuous-time neural networks. *Nature Machine Intelligence*, 2022.
- [8] Makram Chahine, Ramin Hasani, Patrick Kao, Aaron Ray, Ryan Shubert, Mathias Lechner, Alexander Amini, and Daniela Rus. Robust flight navigation out of distribution with liquid neural networks. *Science Robotics*, 2023.
- [9] Mathias Lechner, Ramin Hasani, Alexander Amini, Thomas A. Henzinger, Daniela Rus, and Radu Grosu. Neural circuit policies enabling auditable autonomy. *Nature Machine Intelligence*, 2020.
- [10] Zhaokun Zhou, Yuesheng Zhu, Chao He, Yaowei Wang, Shuicheng Yan, Yonghong Tian, and Li Yuan. Spikformer: When spiking neural network meets transformer. In *International Conference on Learning Representations (ICLR)*, 2023.
- [11] Man Yao, Jiakui Hu, Zhaokun Zhou, Li Yuan, Yonghong Tian, Bo Xu, and Guoqi Li. Spike-driven transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [12] Emre O. Neftci, Hesham Mostafa, and Friedemann Zenke. Surrogate gradient learning in spiking neural networks. *IEEE Signal Processing Magazine*, 2019.
- [13] Friedemann Zenke and Surya Ganguli. Superspike: Supervised learning in multilayer spiking neural networks. *Neural Computation*, 2018.
- [14] Mark Horowitz. Computing’s energy problem (and what we can do about it). In *IEEE International Solid-State Circuits Conference (ISSCC)*, 2014.
- [15] Anthony Brohan, Noah Brown, Justice Carbajal, et al. RT-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [16] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, et al. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024.
- [17] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)*, 2023.

- [18] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023.
- [19] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.
- [20] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- [21] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning (ICML)*, 2024.
- [22] Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control barrier functions: Theory and applications. In *European Control Conference (ECC)*, 2019.
- [23] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.
- [24] Zhiqi Li, Wenhai Wang, Hongyang Li, Enze Xie, Chonghao Sima, Tong Lu, Yu Qiao, and Jifeng Dai. BEVFormer: Learning bird’s-eye-view representation from multi-camera images via spatiotemporal transformers. In *European Conference on Computer Vision (ECCV)*, 2022.
- [25] Alex H. Lang, Sourabh Vora, Holger Caesar, Lubing Zhou, Jiong Yang, and Oscar Beijbom. PointPillars: Fast encoders for object detection from point clouds. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [26] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.
- [27] Michael Laskey, Jonathan Lee, Roy Fox, Anca Dragan, and Ken Goldberg. DART: Noise injection for robust imitation learning. In *Conference on Robot Learning (CoRL)*, 2017.
- [28] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [29] Pim de Haan, Dinesh Jayaraman, and Sergey Levine. Causal confusion in imitation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [30] Chuan Wen, Jierui Lin, Trevor Darrell, Dinesh Jayaraman, and Yang Gao. Fighting copycat agents in behavioral cloning from observation histories. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.